

TypeScript Utils

Miguel Costa
miguelangcosta@gmail.com

Objetivos

- Compilador do TypeScript (`tsconfig.json`)
- Versões do JavaScript (ES6 vs ES2020)
- Módulos, imports e exports
- Interfaces vs types **apenas no contexto de imports e classes**
- TypeScript UtilTypes
- Utilidades que afetam organização e imports

Versões do JavaScript

JavaScript evolui através das versões **ECMAScript (ES)**.

ECMAScript (ES) é o **padrão oficial do JavaScript**, ou seja, a especificação que define como a linguagem deve funcionar.

ES6 (ES2015)

- Introduziu:
 - `import / export`
 - `classes`
 - `let / const`

ES2020

- Inclui tudo do ES6 + recursos mais novos:
 - `optional chaining` `?.`
 - `nullish coalescing` `??`

👉 O TypeScript permite escrever **JavaScript moderno**, mesmo que o código final precise rodar em ambientes mais antigos

Versões do JavaScript

Por que a versão do JavaScript importa no TypeScript?

O TypeScript precisa saber:

- quais recursos do JavaScript podem ser usados
- qual versão de JavaScript deve ser gerada

Isso é definido no **tsconfig.json**

O que é o tsconfig.json?

Arquivo de configuração do compilador TypeScript

Responde perguntas como:

- Quão rigorosas devem ser as verificações?
- Qual versão de JavaScript será gerada?
- Como os módulos devem funcionar?

Criado com:

```
tsc --init
```

Opções principais do tsconfig

```
{  
  "compilerOptions": {  
    "target": "ES2020",  
    "module": "ESNext",  
    "strict": true,  
    "rootDir": "src",  
    "outDir": "dist"  
  }  
}
```

target: versão do JavaScript gerado

"target": "ES2020"

- Controla o **JavaScript de saída**
- Não afeta a sintaxe do TypeScript

Exemplos:

- **ES5** → navegadores antigos
- **ES2020** → Node.js e navegadores modernos

module: como imports são compilados

"module": "ESNext"

Define como **import** / **export** viram JavaScript

Valores comuns:

- **CommonJS** → Node.js mais antigo
 - a. Transforma **import/export** em **require/module.exports**
- **ESNext** → bundlers modernos
 - a. Gera **imports/exports nativos** do ES (JavaScript moderno)
 - b. Usa **import** e **export** sem alteração

```
const x = require("./x");  
exports.y = 1;
```

```
import { x } from "./x.js";  
export const y = 1;
```


O que o **strict** realmente valida

"strict": true

Ativa verificações como:

- propriedades obrigatórias ausentes
- argumentos inseguros em funções
- segurança contra **null** e **undefined**

Sem **strict**:

- muitos erros lógicos passam despercebidos

```
function sum(a: number, b: number) {  
  return a + b;  
}
```

```
const result = sum(5, "10");  
console.log(result);
```

rootDir & outputDir

rootDir: Define **qual é a pasta de origem** do código TypeScript.

outputDir: Define onde o TypeScript vai colocar o JavaScript compilado..

```
{  
  "compilerOptions": {  
    "rootDir": "src",  
    "outDir": "dist"  
  }  
}
```

O que é um módulo no TypeScript?

- Um arquivo se torna um **módulo** quando usa `import` ou `export`
- Cada módulo tem seu próprio escopo

Regra:

- Sem `export` → não reutilizável
- Com `export` → API pública

Por que módulos são importantes?

Módulos oferecem:

- isolamento
- dependências explícitas
- estrutura escalável

Sem módulos:

- variáveis globais vazam
- refatorações são perigosas

Por que módulos são importantes?

1) Isolamento

- Cada arquivo é um módulo com seu próprio escopo.
- Mesmo com o mesmo nome **x**, **não há conflito**, porque cada arquivo tem seu próprio escopo.
- Isso evita bugs quando o projeto cresce.

```
// file1.ts
const x = 10;
export function getX() { return x; }

// file2.ts
const x = 20;
export function getX2() { return x; }
```

Por que módulos são importantes?

2) Dependências explícitas

Quando se usa `import`, está explícito exatamente **de onde vem cada coisa**.

```
import { getX } from "./file1";
```

Isso mostra claramente:

- o que o arquivo precisa
- de onde vem
- o que é usado no projeto

Isso melhora manutenção e entendimento do código.

Por que módulos são importantes?

3) Estrutura escalável

- Quando o projeto cresce, módulos ajudam a organizar.
- Com módulos, cada pasta vira uma “peça” do sistema.

```
src/  
├─ auth/  
│   ├─ auth.service.ts  
│   └─ auth.controller.ts  
├─ user/  
│   ├─ user.service.ts  
│   └─ user.controller.ts  
└─ shared/  
    └─ utils.ts
```

Sem módulos: problemas reais

1) Variáveis globais vazam

Sem módulos, tudo fica no mesmo escopo global

No mundo sem módulos, isso pode sobrescrever e causar bugs.

```
// arquivo1.js
```

```
const x = 10;
```

```
// arquivo2.js
```

```
const x = 20;
```


Sem módulos: problemas reais

2) Refatorações são perigosas

Quando tudo é global, mudar um nome pode quebrar várias partes do código.

Exemplo:

Se você renomeia `getX()` para `getValue()` em um arquivo, pode esquecer de atualizar outros arquivos.

Com módulos, o TypeScript avisa imediatamente:

“Esse import não existe mais”

Named exports

Recomendado como padrão!

Por quê?

- contratos claros
- melhor suporte das ferramentas
- refatorações mais seguras

```
// math.ts
export function soma(a: number, b: number) {
  return a + b;
}

export function subtrai(a: number, b: number) {
  return a - b;
}
```

```
import { soma, subtrai } from "./math";

import { somar } from "./math"; // ✗ erro
```

Default exports

Características:

- apenas um **default** por arquivo
- o nome no **import** é flexível
- Útil quando o ficheiro tem uma **função/classe principal**

```
// logger.ts
export default function log(message: string) {
  console.log(message);
}
```

```
import log from "./logger";
import meuLogger from "./logger";
```

Default vs Named exports

Default exports

- bons para arquivos simples
- responsabilidade única

Contras:

- refatoração mais fraca
- nomes inconsistentes

Named exports

- explícitos
- melhores para trabalho em equipe
- preferidos em bases grandes

Misturando default e named exports

É permitido, mas:

- use com moderação
- aumenta a complexidade mental

```
export default class ApiClient {}  
export const TIMEOUT = 5000;
```

Importando types vs valores de runtime

O TypeScript diferencia:

- **Valores de runtime** - São coisas que **existem no JavaScript final**, depois de compilar.
 - a. **Classes**
 - b. **Funções**
 - c. **Constantes**
 - d. **Objetos**
 - e. **variáveis**
- **Tipos de tempo de compilação** - São coisas que **existem apenas no TypeScript**, e desaparecem quando o código é compilado para JavaScript.
 - a. **Interfaces**
 - b. **Types**

Importando types vs valores de runtime

```
// user.ts
export interface User {
  name: string;
}

export function createUser(name: string): User {
  return { name };
}
```

```
import { createUser } from "./user";           // runtime
import type { User } from "./user";           // compile-time
```

Por que **import type** é importante

- removido do JavaScript final
- evita dependências circulares
- deixa a intenção clara

Boa prática:

- sempre usar para interfaces e types

Utility types que afetam imports

Utilidades comuns:

- `Partial<T>`
- `Readonly<T>`
- `Pick<T, K>`
- `Omit<T, K>`

Usadas para:

- evitar redefinir interfaces
- criar variações de tipos exportados

Utility types - **Partial<T>**

- Transforma **todas as propriedades** de um tipo em **opcionais**.
- Quando usar? Quando se quer fazer atualizações parciais (ex: editar um utilizador).

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type UserUpdate = Partial<User>;
```

UserUpdate fica igual a:

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type UserUpdate = Partial<User>;
```

Utility types - `ReadOnly<T>`

- Transforma **todas as propriedades** em **somente leitura**.
- Quando usar? Quando se quer garantir que o objeto **não seja alterado**.

```
interface User {  
  id: number;  
  name: string;  
}  
  
type UserReadOnly = Readonly<User>;
```

`UserUpdate` fica igual a:

```
{  
  readonly id: number;  
  readonly name: string;  
}
```

Utility types - **Pick<T, K>**

- Cria um novo tipo com **apenas algumas propriedades** do tipo original.
- Quando usar? Quando queres um tipo mais pequeno para uma parte específica do app (ex: lista de utilizadores).

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type UserPreview = Pick<User, "id" | "name">;
```

UserPreview fica igual a:

```
{  
  id: number;  
  name: string;  
}
```

Utility types - `Omit<T, K>`

- Cria um novo tipo **excluindo algumas propriedades** do tipo original.
- Quando usar? Quando queres remover campos sensíveis ou desnecessários (ex: não enviar password).

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type UserWithoutEmail = Omit<User, "email">;
```

`UserWithoutEmail` fica igual a:

```
{  
  id: number;  
  name: string;  
}
```

Utility types

Outros utility types úteis:

- `Required<T>`
- `Record<K, T>`
- `Exclude<T, U>`
- `Extract<T, U>`
- `NonNullable<T>`
- `ReturnType<T>`
- `Parameters<T>`

Barrel files (**index.ts**)

O que é um Barrel File?

- Um **arquivo centralizador de exports**
- Normalmente chamado de **index.ts**
- Reexporta conteúdos de vários arquivos de uma pasta

Por que barrel files são úteis:

- imports mais limpos
- escondem a estrutura de pastas
- facilitam refatorações

Use por feature, não globalmente

Por que usar Barrel Files?

Organizar e simplificar imports

Em vez de importar muitos ficheiros diferentes:

```
import { UserService } from "../user/user.service";  
import { UserController } from "../user/user.controller";  
import { UserDTO } from "../user/user.dto";
```

Basta:

```
import { UserService, UserController, UserDTO } from "../user";
```


Como criar um Barrel File

```
// user/index.ts
export * from "./user.service";
export * from "./user.controller";
export * from "./user.dto";
```

```
user/
├─ index.ts
├─ user.service.ts
├─ user.controller.ts
└─ user.dto.ts
```

Resolução de módulos

Ao importar `./user`, o TypeScript procura:

- `user.ts`
- `user/index.ts`
- `user.d.ts`

`user.ts` é código, `user/index.ts` organiza exports e `user.d.ts` só declara tipos.

Ficheiros de declaração de tipos

Exemplo: `user.d.ts`

É um ficheiro de declarações de tipos

- Só define tipos (interfaces, types, declarações de módulos)
- **Não gera JavaScript**
- Usado para **tipar coisas que não têm tipagem** ou para **tipos globais**

```
// user.d.ts
interface User {
  id: number;
  name: string;
}
```

Este tipo fica disponível sem precisar importar!

Path aliases

Path aliases permitem que uses **atalhos** para importar ficheiros, em vez de escrever caminhos longos relativos.

```
import { UserService } from "../../../../../user/user.service";
```

Com path alias:

```
import { UserService } from "@user/user.service";
```

Path aliases no tsconfig

baseUrl: Define a **pasta base** para resolver **imports absolutos** (path aliases).

É usada junto com **paths**. Exemplo, resolve para : `./src/user/user.service`

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@app/*": ["src/*"],
      "@user/*": ["src/user/*"]
    }
  }
}
```

Agora isto funciona:

```
import { UserService } from "@user/user.service";
```

Erros comuns de config e imports

- Desativar `strict`
- Importar types como valores
- Usar default exports em excesso
- Caminhos relativos profundos

Regras recomendadas

- Mantenha `strict: true`
- Prefira named exports
- Use interfaces como contratos de classe
- Use `import type`
- Configure paths desde o início

Questões